CONTENTS

LIST OF ABBREVIATIONS 2
LIST OF TABLES 3
LIST OF FIGURES 4
INTRODUCTION 5
CHAPTER 1. DATA SECURITY IN CLOUD DATABASES 7
1.1. Cloud computing and managed databases 7
1.2. The shared responsibility model 7
1.3. Threats to cloud databases 8
1.4. Network and transport protection 8
1.5. Authentication and authorisation 9
1.6. Data masking 9
1.7. Encryption at rest, in transit and in use 9
1.8. Key management 10
1.9. Auditing 11
1.10. Comparison of three platforms 11
CHAPTER 2. SECURITY DESIGN FOR THE CLINIC DATABASE 12
2.1. Platform selection and environment 12
2.2. Scenario and data classification 12
2.3. Threat model 13
2.4. System architecture 13
2.5. Roles and privileges 14
2.6. Mapping threats to controls 14
2.7. Field encryption design 15
CHAPTER 3. IMPLEMENTATION AND EVALUATION 16
3.1. Implementation overview 16
3.2. Transport and network protection 16
3.3. Data API exposure 17
3.4. Role-based access control 17
3.5. Row-Level Security 18
3.6. Data masking 19
3.7. Client-side encryption 20
3.8. Encryption at rest 21
3.9. Audit logging 21
3.10. Performance 22
3.11. Evaluation of the controls 23
CONCLUSIONS 24
REFERENCES 25

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AAD	Additional Authenticated Data
AES	Advanced Encryption Standard
API	Application Programming Interface
CIDR	Classless Inter-Domain Routing
DBA	Database Administrator
DDM	Dynamic Data Masking
GCM	Galois/Counter Mode
HMAC	Hash-based Message Authentication Code
IaaS	Infrastructure as a Service
IP	Internet Protocol
KMS	Key Management Service
PaaS	Platform as a Service
RBAC	Role-Based Access Control
RDS	Relational Database Service (Amazon)
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SQL	Structured Query Language
TDE	Transparent Data Encryption
TLS	Transport Layer Security

LIST OF TABLES

Table 1. Team members and task allocation 6

Table 2. Division of security duties for a managed database 7

Table 3. Main threats to a cloud database 8

Table 4. Encryption by data state 10

Table 5. Security features of three cloud database platforms 11

Table 6. Data classification and protection 12

Table 7. Role and privilege matrix 14

Table 8. Threats, controls and verification 14

Table 9. Summary of experiment results 16

Table 10. Query latency, plain and encrypted columns (milliseconds) 22

Table 11. What each control stops and does not stop 23

LIST OF FIGURES

Figure 1. System architecture and trust boundaries 13

Figure 2. Defence-in-depth layers of the design 14

Figure 3. Field encryption, storage and decryption flow 15

Figure 4. Results of the transport and network tests 17

Figure 5. Results of the Data API tests 17

Figure 6. Results of the access control tests 18

Figure 7. Results of the Row-Level Security tests 19

Figure 8. Results of the masking tests 20

Figure 9. Results of the client-side encryption tests 20

Figure 10. Audit log traceability check and sample entries 22

INTRODUCTION

Moving a database to the cloud changes who can touch the data. On a private server, the organisation controls the hardware, the operating system and the network. On a managed cloud service, the provider operates these layers, the database is reachable over the public Internet unless restricted, and new interfaces such as automatically generated web APIs may appear beside the classic database port. A wrong setting can therefore expose data to anyone on the Internet rather than to people inside one office.

Cloud providers advertise strong built-in protection, yet this protection covers only part of the problem. The shared responsibility model [1], [2] leaves data, access rights and configuration with the customer. Encryption at rest, for example, does nothing against a user who logs in with valid credentials. A useful study of cloud database security must therefore ask two questions of every mechanism: which threat does it stop, and which threat does it leave open?

Objectives. This report has three objectives: to explain the main threats and security mechanisms for cloud databases; to implement a layered set of mechanisms on a real cloud database; and to verify by experiment what each mechanism prevents, what it does not prevent and how much it costs in performance.

Scope. The implementation uses a PostgreSQL database hosted on the Free plan of Supabase, for a fictional clinic with 200 synthetic patient records. Supabase was selected because the project could be created immediately and without a payment card. Azure SQL Database and Amazon RDS are covered through their documentation and are compared in Chapter 1, but they were not deployed. No real personal or medical data was used, and the system is an academic demonstration rather than a product ready to hold real patient records.

Method. Each threat was mapped to a control, the control was configured with repeatable scripts, and an automated test attempted both a legitimate action and an unauthorised one against the live database. The database was then deleted and rebuilt from the scripts, and all figures in Chapter 3 are drawn from the output of this clean run, stored in the `evidence` folder of the project.

Structure. Chapter 1 presents the background and compares three platforms. Chapter 2 describes the scenario, threat model and design. Chapter 3 reports the implementation, the results and the limitations. The Conclusions summarise the findings and suggest further work.

Team and task allocation. The project was carried out by a team of three members. Table 1 shows the part of the work each member was responsible for.

Table 1. Team members and task allocation

Member	Student ID	Responsibilities
Full name A	ID	Cloud and monitoring: Supabase environment; transport, network and Data API protection; encryption at rest; audit logging; performance measurement (Sections 3.2, 3.3, 3.8, 3.9, 3.10)
Full name B	ID	Database security: schema and synthetic data; roles, RLS, masking and client-side encryption; test scenarios (Chapter 2; Sections 3.4 to 3.7)
Full name C	ID	Theory and integration: Introduction, Chapter 1, Conclusions and References; report assembly and formatting; slides; cross-checking of results

CHAPTER 1. DATA SECURITY IN CLOUD DATABASES

1.1. Cloud computing and managed databases

NIST defines cloud computing as a model for convenient, on-demand network access to a shared pool of configurable computing resources that can be provisioned rapidly with minimal management effort [3]. The definition lists five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service. It also describes three service models, namely IaaS, PaaS and SaaS.

A managed database service is close to the PaaS model. The customer receives a working database endpoint, while the provider handles hardware, operating system, patching and basic operations. Two of the characteristics above matter directly for security. Broad network access means that the database may be reachable from anywhere, so the network is no longer a natural barrier. Resource pooling means that customers share physical infrastructure and depend on the isolation provided by the operator. NIST SP 800-144 accordingly advises organisations to understand the security of the provider environment and to keep accountability for their own data when they use a public cloud [4].

1.2. The shared responsibility model

Both AWS and Microsoft describe cloud security as a division of duties [1], [2]. AWS summarises it as security "of" the cloud, which belongs to the provider, and security "in" the cloud, which belongs to the customer. Table 2 applies this idea to a managed database. The key observation is that the entries in the right-hand column cannot be delegated: a provider cannot decide which clinic employee may read which patient record.

Table 2. Division of security duties for a managed database

Provider	Customer
Data centres, hardware and physical security	Which data is stored, and how sensitive it is
Host operating system and database software operation	Accounts, roles, privileges and row-level rules
Isolation between tenants	Network allowlists and the interfaces that are exposed
Infrastructure of the platform, including storage encryption where it is offered	Protection of application keys, secrets and access tokens

Provider	Customer
Availability of platform logging	Enabling audit, exporting logs and reviewing them

1.3. Threats to cloud databases

OWASP ranks broken access control as the first risk in its 2021 Top 10 and cryptographic failures as the second [5], [6]. Both are central to databases. Table 3 lists the threats considered in this report, grouped by who the attacker is and how the attacker reaches the data. The list is not exhaustive: denial of service, for example, is outside the scope of this study.

Table 3. Main threats to a cloud database

Threat	Description	Possible impact
Open network access	The database endpoint accepts connections from any address	Password guessing, exploitation
Eavesdropping	Traffic is read or altered between client and server	Credentials and data disclosed
Exposed web interface	An automatically generated API publishes tables	Public reading of data
Excessive privilege	A user or application has more rights than the job needs	Wider damage from one mistake or stolen account
Horizontal access	A valid user reads records of other users	Privacy breach inside the organisation
Privileged insider	An administrator or provider operator reads data directly	Disclosure that normal access rules do not stop
Stolen storage or backup	Disks or backup copies leave the controlled environment	Offline reading of all data
Undetected activity	Actions leave no reviewable record	Incidents are found late or never

1.4. Network and transport protection

Network controls decide who may even attempt to connect. An IP allowlist reduces the attack surface, although it does not authenticate anyone and it applies only to the interfaces it covers. In Supabase, for example, network restrictions apply to direct and pooled Postgres connections but not to the REST API, Storage or Auth services [7].

TLS protects traffic against reading and modification on the network. Two points deserve attention. First, the setting must be enforced by the server: Supabase leaves SSL optional for direct Postgres connections unless the

administrator enforces it [8]. Second, encryption alone does not prove who the server is. To resist an active man-in-the-middle, the client must also verify the server certificate, which PostgreSQL clients express as the `verify-full` mode [8].

1.5. Authentication and authorisation

Authentication establishes who a user is, and authorisation decides what that user may do. The principle of least privilege requires that each identity has only the rights its job needs, and access should be denied by default [5]. PostgreSQL implements RBAC through roles and object privileges granted with `GRANT` [9]. Privileges can be set per table and even per column.

Table privileges are all-or-nothing for rows, so PostgreSQL adds RLS. Once RLS is enabled on a table, normal access must be allowed by a policy, and if no policy exists the default is to deny [10]. A `USING` clause selects which existing rows are visible, and a `WITH CHECK` clause validates new row values. The bypass rules are important: superusers and roles with the `BYPASSRLS` attribute always bypass RLS, and table owners bypass it unless `FORCE ROW LEVEL SECURITY` is set [10]. RLS therefore controls ordinary users, not administrators. SQL Server and Azure SQL offer an equivalent through security policies with filter and block predicates [11].

1.6. Data masking

Masking hides part of a value in query results, for instance by showing only the last digits of a phone number. It is a convenience control, not encryption, because the original value stays unchanged in storage. Microsoft states that dynamic data masking does not aim to stop users running exhaustive queries, and warns that users with ad hoc query rights can infer real values: a user can filter on a masked column with a guessed range and identify the rows that match, even though the output shows zeros [12]. Masking should thus be combined with access control and auditing.

1.7. Encryption at rest, in transit and in use

Encryption protects data in three states, and each state answers a different threat. Table 4 summarises them.

Table 4. Encryption by data state

State	Typical mechanism	What it does not stop
At rest	Storage encryption, TDE	Anyone who queries through the running database, because data is decrypted for them
In transit	TLS	Access by a legitimate endpoint; attacks on the client or the server
Against the server	Client-side encryption such as Always Encrypted	Compromise of the client or its keys; it also restricts queries on the encrypted columns

TDE on Azure SQL encrypts the database, its backups and its transaction logs at the page level. Each page is decrypted when read into memory and is encrypted before being written to disk, and TDE is on by default for new databases [13]. The same source notes that an exported BACPAC file is not encrypted, which shows how protection can end at a boundary that users do not notice. Amazon RDS similarly uses AES-256 for storage, logs, automated backups, read replicas and snapshots, and it can only be turned on when an instance is created [14].

Client-side encryption goes further by keeping the keys away from the database engine. Always Encrypted separates those who own the data from those who manage it, including cloud database operators, with the cost that operations on encrypted columns are limited [15]. Deterministic encryption allows equality lookups but can leak patterns, while randomised encryption is stronger and does not allow searching. The design in this report faces the same trade-off.

The implementation uses AES [16] in GCM mode, a NIST-recommended algorithm for authenticated encryption with associated data [17]. Authenticated encryption matters because it detects modification as well as hiding content. For exact-match search over encrypted values, a keyed hash such as HMAC [18] can serve as a blind index: equal inputs give equal outputs, while a reader without the key cannot compute them.

1.8. Key management

Encryption moves the problem from protecting data to protecting keys. NIST SP 800-57 gives guidance on the entire key lifecycle, including generation, storage, use, rotation and destruction [19]. Cloud platforms offer a choice of ownership. With provider-managed keys, operation is simple, but trust in the

provider remains. With customer-managed keys, as in Azure Key Vault integration for TDE [13] or customer managed keys in AWS KMS [14], the customer can control rotation and access and can make the data unreadable by revoking the key. This control carries its own risk, because losing the key makes data inaccessible.

1.9. Auditing

Preventive controls can fail, so a system also needs a record of what happened. Azure SQL auditing writes database events to a storage account, a Log Analytics workspace or Event Hubs [20]. Amazon RDS offers Database Activity Streams for near real-time monitoring [21]. In PostgreSQL, the pgAudit extension produces detailed session and object audit logs through the standard logging facility [22]. Supabase allows pgAudit configuration only at the role level and shows the results in its log viewer [23]. A log is useful only if it is retained long enough and reviewed by someone, so retention must be treated as part of the design.

1.10. Comparison of three platforms

Table 5 compares the features relevant to this study. It relies on vendor documentation. Azure SQL offers the most ready-made features, while Supabase requires more to be built by hand. The results in Chapter 3 say nothing about the Azure and AWS features themselves.

Table 5. Security features of three cloud database platforms

Feature	Azure SQL Database	Amazon RDS for PostgreSQL	Supabase (PostgreSQL)
Encryption at rest	TDE, on by default for new databases	AES-256, chosen when the instance is created	AES-256 stated by the provider [24]
Customer-managed key	Yes, Azure Key Vault	Yes, AWS KMS	No option found in the pages reviewed
Transport	TLS	SSL/TLS [25]	TLS; enforcement is a setting
Client-side encryption	Always Encrypted built in	Implemented by the application	Implemented by the application
Row-level rules	Security policies	PostgreSQL RLS	PostgreSQL RLS
Masking	DDM built in	Views or similar	Views or similar
Audit	SQL auditing	pgAudit, Activity Streams	pgAudit at role level

CHAPTER 2. SECURITY DESIGN FOR THE CLINIC DATABASE

2.1. Platform selection and environment

The project was first planned on Azure SQL Database, but registration for the student subscription was not successful. Supabase was then adopted since it provides a real PostgreSQL service on the Internet with no payment card required at sign-up. The Free plan has clear limits: 500 MB per project, one day of log retention, no automatic backups and a pause after one week of inactivity [26]. These limits shaped the design, in that logs had to be exported immediately and the whole database had to be reproducible from scripts.

The database ran PostgreSQL 17.11 with pgAudit 17.1 in region `ap-southeast-2`. On the Free plan, the direct connection supports only IPv6, so the client connected through the shared pooler in session mode on port 5432, which also supports custom database roles [27]. Scripts were written in Node.js with the `pg` driver.

2.2. Scenario and data classification

The scenario is a small fictional clinic. Doctors treat their own patients, a receptionist manages appointments and contacts patients, a database administrator keeps the system running, and an auditor checks that rules are being followed. All 200 patient records are synthetic: phone numbers start with `000` and national identity numbers start with `999`, which makes them recognisably fake, and email addresses use the reserved domain `example.com`. Table 6 shows how each kind of data is classified and protected.

Table 6. Data classification and protection

Data	Sensitivity	Protection
National ID	High	AES-256-GCM on the client, HMAC blind index for lookup
Diagnosis	High	AES-256-GCM on the client
Phone, email	Medium	Masked for the receptionist, visible to the treating doctor
Patient name	Medium	Access control and RLS; visible to the receptionist, who needs it
Appointments	Medium	RLS for doctors, read access for the receptionist

2.3. Threat model

The design assumes that an external attacker can reach the public endpoints and may obtain the public anonymous API key, which Supabase projects use in client applications. It assumes that insiders hold valid accounts and may try to exceed their role, and that a database administrator or provider operator may read anything stored in the database. The client machine that holds the encryption keys is trusted. An attacker who controls that machine, or the Supabase account itself, is outside the model, and no mechanism in this design would stop such an attacker.

2.4. System architecture

Figure 1 shows the architecture. The client is the only component that holds encryption keys. All database traffic passes through the shared pooler over TLS. The Data API remains part of the platform, but the `clinic` schema is deliberately not exposed to it. Logs leave the platform through the management interface and are saved as evidence, since the Free plan discards them after one day.

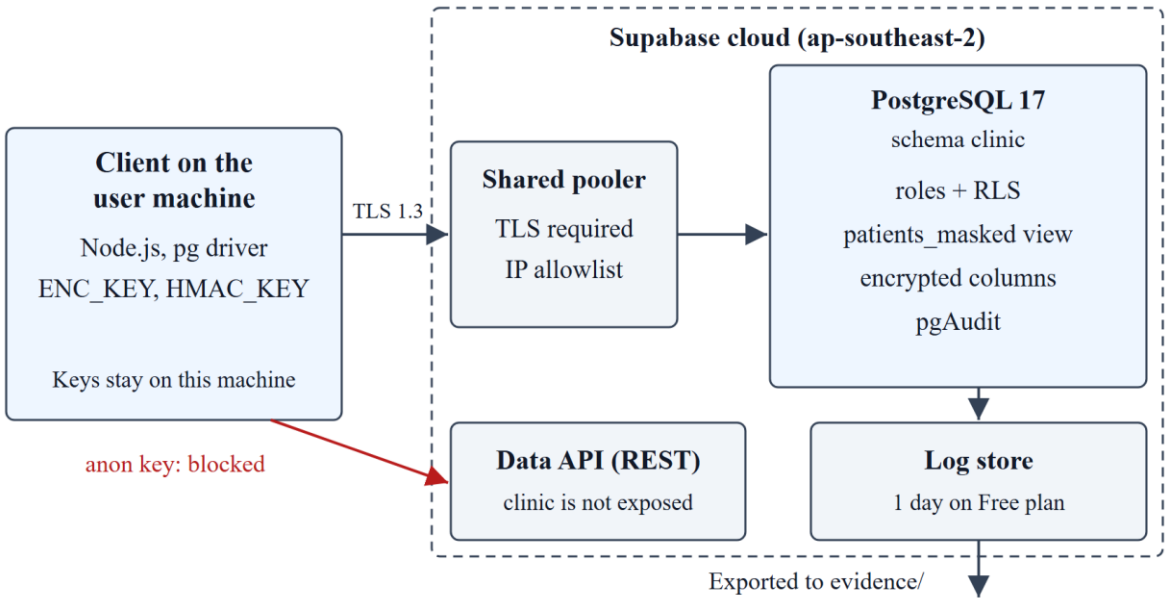


Figure 1. System architecture and trust boundaries

Security is organised as defence in depth, so that no single control is relied on alone. Figure 2 shows the layers used. A request must pass each outer layer before reaching the data, and even an actor who reaches the stored values sees only ciphertext for the most sensitive columns.

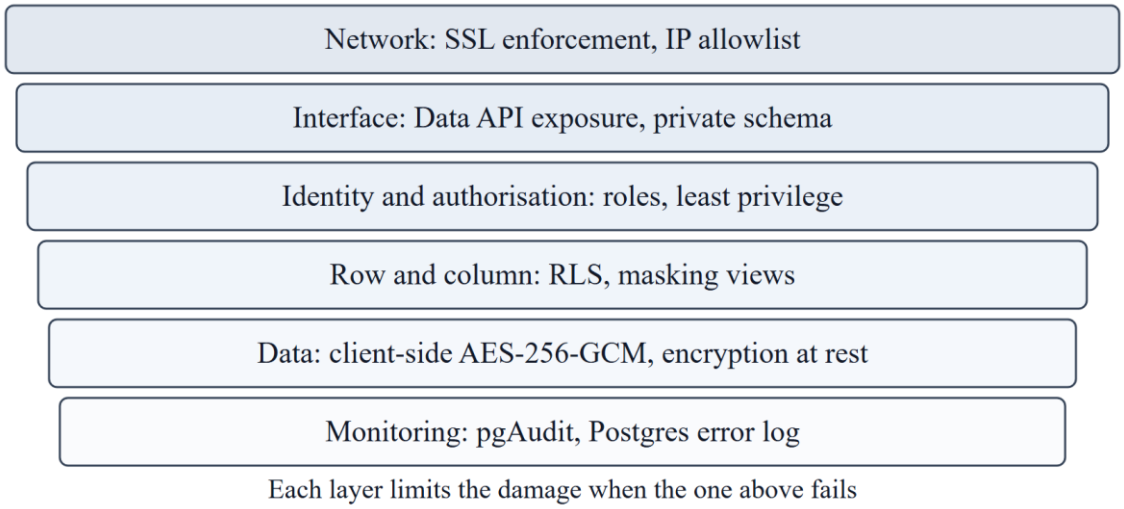


Figure 2. Defence-in-depth layers of the design

2.5. Roles and privileges

Five login roles represent the people in the scenario. Table 7 is the access matrix that the scripts implement. Two choices in this matrix deserve explanation. The receptionist has no privilege on the base `patients` table and reads it only through a masking view. The administrator can read every row, because that job needs visibility for maintenance, yet can modify nothing and holds no encryption key, so the highest-sensitivity columns stay unreadable to this role.

Table 7. Role and privilege matrix

Role	patients table	patients_masked	appointments	Holds key
doctor_a, doctor_b	Own rows: read; update diagnosis	No	Own rows: read	Yes, in the client
receptionist	Denied	Read	Read	No
db_admin	All rows: read	No	All rows: read	No
auditor	Denied	No	Denied	No

2.6. Mapping threats to controls

Table 8 links each threat from Table 3 to the control selected and the test that verifies it. Controls that could not be tested by the customer are marked as such, so that documentation is not presented as proof.

Table 8. Threats, controls and verification

Threat	Control	Verification
Open network access	Network Restrictions	Test T1.4
Eavesdropping	SSL enforcement	Tests T1.1 to T1.3
Exposed web interface	Private schema, no anon grants	Tests T2.1 and T2.2

Threat	Control	Verification
Excessive privilege	RBAC, least privilege	Tests T3.1 to T3.5
Horizontal access	RLS	Tests T4.1 to T4.6
Unneeded disclosure to staff	Masking view	Tests T5.1 to T5.3
Privileged insider	Client-side AES-256-GCM	Tests T6.1 to T6.4
Stolen storage or backup	Provider encryption at rest	Provider statement only; not testable
Undetected activity	pgAudit and Postgres error log	Log export and traceability check

2.7. Field encryption design

Sensitive fields are encrypted by the client before they are sent, as shown in Figure 3. Each value uses AES-256-GCM with a fresh random 96-bit nonce, and the stored value is the nonce, the authentication tag and the ciphertext. The additional authenticated data has the form `patient:id:column`, which binds a ciphertext to its row and column. Without this binding, an insider could copy a diagnosis from one patient to another, and decryption would still succeed. The national ID also receives an HMAC-SHA-256 blind index under a separate key, which permits exact lookup and nothing else.

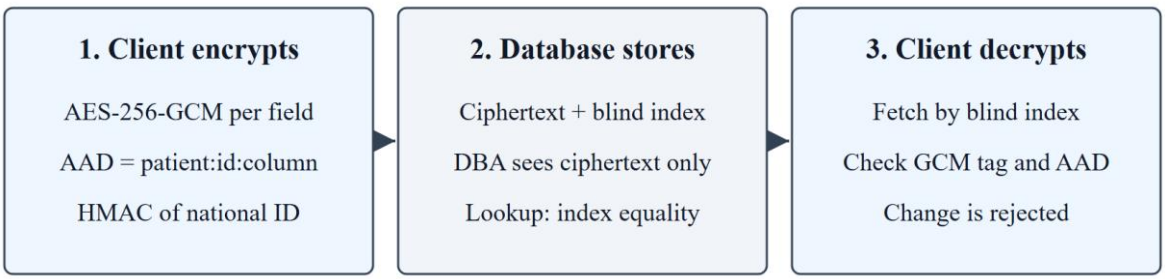


Figure 3. Field encryption, storage and decryption flow

CHAPTER 3. IMPLEMENTATION AND EVALUATION

3.1. Implementation overview

The implementation consists of two SQL files and a small set of Node.js scripts: `configure-cloud.js` sets platform options, `setup-db.js` rebuilds the schema and data, `run-tests.js` executes the experiments, `export-logs.js` collects the logs and `benchmark.js` measures performance. Passwords and keys are generated by the scripts and stored only in a local `.env` file that is excluded from version control. Output is filtered to remove the project identifier, credentials and IP addresses before being written as evidence.

The test script connects to the live database as each role in turn, and a test passes only if the observed behaviour matches the expected one. In the final clean run, **24 of 24** tests passed. Table 9 summarises them by group. One of these, T4.6, only records a fact for the limitations analysis rather than testing a control. A separate suite of seven unit tests for the encryption functions also passed.

Table 9. Summary of experiment results

Group	Control	Tests	Result
T1	Transport and network	4	All passed
T2	Data API exposure	2	All passed
T3	RBAC and least privilege	5	All passed
T4	Row-Level Security	6	All passed
T5	Masking	3	All passed
T6	Client-side encryption	4	All passed

3.2. Transport and network protection

SSL enforcement was turned on through the Management API. As Figure 4 shows, the client negotiated TLS 1.3 (T1.1), and a connection without TLS was refused with an error stating that SSL is required (T1.2). To test the allowlist, the script temporarily replaced it with `203.0.113.0/24`, an address range reserved for documentation, and the next connection attempt was rejected because the client address was not on the list (T1.4). The original settings were restored automatically, and later tests confirmed that connections worked again.

```
[PASS] T1.1 Client connection uses TLS 1.2 or newer
    negotiated protocol: TLSv1.3
[PASS] T1.2 Unencrypted connection is rejected
    rejected: (ESSLREQUIRED) SSL connection is required for user: postgres
[PASS] T1.3 SSL enforcement reported as active
    API: {"currentConfig":{"database":true},"appliedSuccessfully":true}
[PASS] T1.4 IP outside the allowlist is blocked
    blocked: (EADDRNOTALLOWED) address not in tenant allow_list: <ip>
```

Figure 4. Results of the transport and network tests

The result has two limits. The negotiated version describes only the segment from the client to the pooler, as the segment from the pooler to the database cannot be observed from outside. In addition, the test client did not verify the server certificate, so these tests prove encryption but not server identity. A production client should use `verify-full` with the CA certificate of the provider [8].

3.3. Data API exposure

Supabase publishes a REST interface automatically, and network restrictions do not cover it [7], so this interface needed its own test. The clinic tables were placed in a dedicated schema that the Data API does not expose, and all privileges of the `anon` role were revoked. As Figure 5 shows, a request with the anonymous key for the clinic schema failed with HTTP 406 and the error `Invalid schema: clinic` (T2.1).

A control experiment gives this result its meaning. The script created a temporary table in the `public` schema, granted it to `anon` without RLS, and read it through the same API with HTTP 200 (T2.2), after which the table was removed. This demonstrates that the path works and that the earlier refusal was caused by the configuration. It also shows how easily data can be published: one `GRANT` on an exposed table without RLS is sufficient [28].

```
[PASS] T2.1 Anon key cannot read clinic.patients
    HTTP 406 {"code":"PGRST106","details":null,"hint":"Only the following schemas are expose
    d: public, graphql_public","message":"Invalid schema: clinic"}
[PASS] T2.2 Control: a public table granted to anon without RLS is readable
    HTTP 200 [{"note":"synthetic canary row"}]
```

Figure 5. Results of the Data API tests

3.4. Role-based access control

All privileges were first revoked and then granted per role, as in Table 7. Figure 6 shows the receptionist and auditor denied on the patient table, the doctor unable to delete and the administrator unable to update, each with PostgreSQL

error 42501. In test T3.5, the auditor confirmed these rights through catalog functions without reading any clinical data.

```
[PASS] T3.1 Receptionist denied on base patient table
    role=receptionist
    sql=SELECT * FROM clinic.patients
    result: denied, 42501 permission denied for table patients
[PASS] T3.2 Doctor cannot delete patients
    role=doctor_a
    sql=DELETE FROM clinic.patients WHERE patient_id = 2
    result: denied, 42501 permission denied for table patients
[PASS] T3.3 DBA cannot modify clinical data
    role=db_admin
    sql=UPDATE clinic.patients SET phone = 'x'
    result: denied, 42501 permission denied for table patients
[PASS] T3.4 Auditor denied on clinical data
    role=auditor
    sql=SELECT * FROM clinic.patients
    result: denied, 42501 permission denied for table patients
```

Figure 6. Results of the access control tests

3.5. Row-Level Security

RLS was enabled and forced on the `patients` and `appointments` tables. The doctor policy compares each row with the doctor record of the current role:

```
CREATE POLICY doctor_own_patients ON clinic.patients
  FOR SELECT TO doctor_a, doctor_b
  USING (doctor_id = (SELECT doctor_id FROM clinic.doctors WHERE db_role =
current_user));
```

As Figure 7 shows, the same query returned 100 rows only of doctor 1 to `doctor_a`, and 100 rows only of doctor 2 to `doctor_b`. A direct read of a patient of the other doctor returned no rows, and an update attempt on such a patient affected no rows. Neither case raised an error, which matches the documented behaviour of `USING` filters [10]: an unauthorised user cannot even tell whether the record exists.

```
[PASS] T4.1 Doctor A sees only own 100 patients
    role=doctor_a
    sql=SELECT count(*)::int AS visible, min(doctor_id) AS min_doc, max(doctor_id) AS max_do
    c FROM clinic.patients
    result: allowed, 1 rows, sample=[{"visible":100,"min_doc":1,"max_doc":1}]
[PASS] T4.2 Doctor B sees only own 100 patients
    role=doctor_b
    sql=SELECT count(*)::int AS visible, min(doctor_id) AS min_doc, max(doctor_id) AS max_do
    c FROM clinic.patients
    result: allowed, 1 rows, sample=[{"visible":100,"min_doc":2,"max_doc":2}]
[PASS] T4.3 Doctor A cannot read a patient of doctor B
    role=doctor_a
    sql=SELECT patient_id FROM clinic.patients WHERE patient_id = 1
    result: allowed, 0 rows, sample=[]
[PASS] T4.4 Doctor A update on a patient of doctor B affects 0 rows
    role=doctor_a
    sql=UPDATE clinic.patients SET diagnosis_enc = 'tampered' WHERE patient_id = 1
    result: allowed, 0 rows, sample=[]
```

Figure 7. Results of the Row-Level Security tests

Test T4.6 recorded that the platform role `postgres` has `rolbypassrls = true`. Therefore, RLS provides no protection against whoever holds that password. This finding is the reason why RLS alone was not considered enough for the most sensitive columns.

3.6. Data masking

PostgreSQL has no built-in equivalent of DDM, so masking was implemented with a view:

```
CREATE VIEW clinic.patients_masked AS
SELECT patient_id,
       full_name,
       '*******' || right(phone, 3)                                  AS phone,
       left(email, 1) || '***@' || split_part(email, '@', 2)         AS email,
       doctor_id
FROM clinic.patients;
```

The receptionist holds a privilege only on this view. As Figure 8 shows, phone and email appear masked (T5.1), and the doctor still sees the real phone number of an own patient (T5.3). Test T5.2 examined the inference attack described for DDM [12]. Filtering on the real phone prefix `000` returned no rows, since the condition is applied to the masked output rather than to the stored value.

```
[PASS] T5.1 Receptionist sees masked phone and email
    role=receptionist
    sql=SELECT * FROM clinic.patients_masked ORDER BY patient_id LIMIT 3
    result: allowed, 3 rows, sample=[{"patient_id":1,"full_name":"Nguyen Hai Giang","phone":
    "*******181","email":"p***@example.com","doctor_id":2},{"patient_id":2,"full_name":"Nguy
    en Duc Chi","phone":"*******941","email":"p***@example.com","doctor_id":1},{"patient_id"
    :3,"full_name":"Nguyen Hai Trang","phone":"*******743","email":"p***@example.com","docto
    r_id":2}]
[PASS] T5.2 Filtering on the masked column does not reveal the real value
    role=receptionist
    sql=SELECT patient_id FROM clinic.patients_masked WHERE phone LIKE '000%'
    result: allowed, 0 rows, sample=[]
[PASS] T5.3 Doctor sees the real phone of an own patient
    role=doctor_a
    sql=SELECT patient_id, phone FROM clinic.patients ORDER BY patient_id LIMIT 1
    result: allowed, 1 rows, sample=[{"patient_id":2,"phone":"0007256941"}]
```

Figure 8. Results of the masking tests

This result should not be overstated. It covers one filtering attempt, not a full analysis of inference attacks. The design also depends entirely on denying the base table, since a single mistaken `GRANT` would remove the protection. Moreover, the view runs with the rights of its owner, so RLS does not limit the receptionist, who can see all patients, as this role requires.

3.7. Client-side encryption

Figure 9 shows what the administrator sees: national ID and diagnosis are only Base64 ciphertext, even though this role can read every row (T6.1). A client with the key recovered the values (T6.2). A change of a single bit made decryption fail (T6.3), and so did reading a ciphertext as if it belonged to another patient (T6.4). These two tests show that the design provides integrity as well as confidentiality, because the error is detected at the client, even when the attacker has full control of the stored data.

```
[PASS] T6.1 DBA reads only ciphertext of sensitive columns
    role=db_admin
    sql=SELECT patient_id, national_id_enc, diagnosis_enc FROM clinic.patients WHERE patient
    _id = 2
    result: allowed, 1 rows, sample=[{"patient_id":2,"national_id_enc":"HBfGHIaxtrDFIZ2aG/aP
    2+aDxmJCsD/I9H+G9Defh6ghNzCEc6I43A==","diagnosis_enc":"MSZdgN3KXWVOZ3o0qy8TKPDBgZ0oAUH8c
    3SKkkudbmqWbXW4SoZwY15lBA=="}]
[PASS] T6.2 Client holding the key recovers the plaintext
    patient 2 diagnosis decrypted: Type 2 diabetes; national id format valid: true
[PASS] T6.3 Modified ciphertext is detected
    decryption refused: Unsupported state or unable to authenticate data
[PASS] T6.4 Ciphertext copied to another row is detected
    decryption refused: Unsupported state or unable to authenticate data
```

Figure 9. Results of the client-side encryption tests

The limits are significant. The database cannot sort, filter by range or search within the encrypted columns. The blind index reveals which patients share the same national ID value and supports only exact matching. For this demonstration, a single key is shared by all doctors and kept in a local file, with no rotation, so a stolen file exposes every record. A real system would keep keys in a KMS, as in the lifecycle described in NIST SP 800-57 [19].

3.8. Encryption at rest

Supabase states that all customer data is encrypted at rest with AES-256 [24]. A customer of a managed service has no access to the underlying disks and no query that proves this property. This control was therefore **not verified** in this project, and this report records it as a statement by the provider. This is itself a practical consequence of shared responsibility: some controls can only be accepted on trust, or through external audit reports. Client-side encryption reduces the importance of this trust for the two columns it covers, but not for names, phone numbers and emails.

3.9. Audit logging

pgAudit was enabled for each demonstration role with the classes `read, write, ddl, role`. After the tests, the logs were exported through the Management API. Figure 10 shows the traceability check: every attempted unauthorised action, and every sampled sensitive read, was found with the correct role name. The export covers two complete test runs and contains 20 pgAudit entries and 8 permission-denied entries, which means 10 and 4 per run.

```
20 pgaudit entries, 8 permission-denied entries

Traceability
FOUND    denied  receptionist  SELECT * FROM clinic.patients
FOUND    denied  doctor_a      DELETE FROM clinic.patients
FOUND    denied  db_admin      UPDATE clinic.patients
FOUND    denied  auditor       SELECT * FROM clinic.patients
FOUND    audit   doctor_a      UPDATE clinic.patients SET diagnosis_enc = 'tampered'
FOUND    audit   receptionist  clinic.patients_masked
FOUND    audit   db_admin      national_id_enc

Entries
2026-11-07 06:33:42.200000000  user=receptionist  ERROR  42501
    permission denied for table patients
    query: SELECT * FROM clinic.patients
2026-11-07 06:33:44.158000000  user=doctor_a  ERROR  42501
    permission denied for table patients
    query: DELETE FROM clinic.patients WHERE patient_id = 2
```

Figure 10. Audit log traceability check and sample entries

The experiment produced one useful finding. The denied statements appeared in the Postgres error log but not as pgAudit entries, because each run produced exactly 10 pgAudit entries for 10 successful statements. An audit process that reads only `AUDIT:` lines would therefore miss exactly the attempts it most needs to see. The audit design must include both sources. Other limits also apply: the `postgres` role was not configured for auditing in this demonstration, statement parameters were not logged, and logs last only one day on this plan unless exported.

3.10. Performance

The performance test compared plain and encrypted columns using two tables of 200 rows, on one connection. Each operation ran 100 times after 10 warm-up runs, and plain and encrypted operations were interleaved so that network changes would affect both alike. Overhead is calculated from the median times:

$$Overhead = \frac{t_{enc} - t_{plain}}{t_{plain}} \times 100\%$$

Table 10. Query latency, plain and encrypted columns (milliseconds)

Operation	Median plain	Median enc.	p95 plain	p95 enc.
Lookup by national ID	236.86	236.88	237.71	237.62
Read 200 rows	237.54	242.07	238.10	479.11
Insert 1 row, then delete	475.43	475.71	476.41	476.77

According to Table 10, median overhead was 0.01% for lookup, 1.9% for reading 200 rows and 0.06% for insert. The network dominates these figures. A single query costs about 237 ms, and the insert test costs about twice that because it issues two statements. Local encryption and decryption together took only 0.0187 ms per value. This explains why the row read, which decrypts 400 values, is the only case with a visible difference.

The p95 for reading encrypted rows rose to 479.11 ms, about one additional network round trip. The larger encrypted result may be the reason, but this cause was not investigated, so the figure is reported as observed. The test used one client, a small table and a shared Free plan service in a distant region. It shows that the cost of encryption was small in this setting, but it should not be read as a prediction for a production workload or a server close to the client.

3.11. Evaluation of the controls

Table 11 brings the results together and is the main finding of this report. No control covers every threat, and each one has a clear point at which another control must take over.

Table 11. What each control stops and does not stop

Control	Stops	Does not stop
SSL enforcement	Plaintext connections	Impersonation of the server unless the client verifies the certificate
IP allowlist	Database connections from other addresses	The REST API; attackers on an allowed network
Private schema	Access through the Data API	Direct database connections with valid credentials
RBAC	Actions beyond the role	Misuse of rights the role legitimately holds
RLS	Reading and updating rows of other doctors	Roles with `BYPASSRLS`, such as `postgres`
Masking view	Casual disclosure to staff	Anyone granted the base table; visible last digits and names
Client-side encryption	Reading or undetected change by DBA and platform	Compromise of the client or keys; column search and sorting
Encryption at rest	Offline reading of storage (as stated by the provider)	Any access through the running database
pgAudit and error log	Undetected activity, when reviewed	The action itself; it records and does not block

CONCLUSIONS

This project studied the threats to cloud databases and implemented a layered set of controls on a real PostgreSQL service, using a fictional clinic with synthetic data. Network and transport controls, API exposure settings, RBAC, RLS, view-based masking, client-side encryption and audit logging were all configured through repeatable scripts. In the final run from a clean rebuild, 24 of 24 automated tests passed, and every unauthorised attempt was traced to the correct role in the logs.

Three lessons stand out. First, the customer side of shared responsibility is large: the exposures demonstrated in this study, such as a table published through the REST API by one `GRANT`, come from configuration and not from provider failure. Second, each control has a specific limit. RLS does not bind the platform administrator, an allowlist does not cover the REST API, and encryption at rest does not stop a valid login. Security comes from the combination of layers, and Table 11 shows which layer covers which gap. Third, tests can show things that documentation does not make obvious, as with the denied statements that appeared in the error log rather than in pgAudit.

Client-side encryption was the strongest control against privileged insiders, and its measured median overhead was at most 1.9% in this setting, where network latency dominated. Its real cost lies in functionality and key management rather than in speed.

Limitations. Only one platform was deployed, so the comparison with Azure and AWS relies on documentation. Encryption at rest could not be verified. The client did not verify the server certificate. Keys were stored in a local file with no rotation, and the audit configuration excluded the `postgres` role. The performance test was small and used a distant server. The inference test of the masking view covered a single attempt.

Future work. The most useful next steps are to verify server certificates with `verify-full`, to move keys to a KMS with rotation and a separate key per doctor, to ship logs to an external store that administrators cannot alter, and to repeat the experiments on Azure SQL so that its built-in Always Encrypted, DDM and TDE features can be compared directly with the hand-built equivalents used here.

REFERENCES

[1] Amazon Web Services, "Shared Responsibility Model", https://aws.amazon.com/compliance/shared-responsibility-model/, accessed 7 November 2026.

[2] Microsoft, "Shared responsibility in the cloud", https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility, accessed 7 November 2026.

[3] P. Mell and T. Grance, "The NIST Definition of Cloud Computing", NIST SP 800-145, NIST, 2011.

[4] W. Jansen and T. Grance, "Guidelines on Security and Privacy in Public Cloud Computing", NIST SP 800-144, NIST, 2011.

[5] OWASP, "A01:2021 – Broken Access Control", https://top10.owasp.org/2021/A01_2021-Broken_Access_Control/, accessed 7 November 2026.

[6] OWASP, "A02:2021 – Cryptographic Failures", https://top10.owasp.org/2021/A02_2021-Cryptographic_Failures/, accessed 7 November 2026.

[7] Supabase, "Network Restrictions", https://supabase.com/docs/guides/platform/network-restrictions, accessed 7 November 2026.

[8] Supabase, "Postgres SSL Enforcement", https://supabase.com/docs/guides/platform/ssl-enforcement, accessed 7 November 2026.

[9] PostgreSQL Global Development Group, "Privileges", https://www.postgresql.org/docs/current/ddl-priv.html, accessed 7 November 2026.

[10] PostgreSQL Global Development Group, "Row Security Policies", https://www.postgresql.org/docs/current/ddl-rowsecurity.html, accessed 7 November 2026.

[11] Microsoft, "Row-level security", https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security, accessed 7 November 2026.

[12] Microsoft, "Dynamic data masking", https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking, accessed 7 November 2026.

[13] Microsoft, "Transparent data encryption – Azure SQL Database and Azure SQL Managed Instance", https://learn.microsoft.com/en-us/azure/azure-

sql/database/transparent-data-encryption-tde-overview, accessed 7 November 2026.

[14] Amazon Web Services, "Encrypting Amazon RDS resources", https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.Encryption.html, accessed 7 November 2026.

[15] Microsoft, "Always Encrypted", https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-database-engine, accessed 7 November 2026.

[16] NIST, "Advanced Encryption Standard (AES)", FIPS 197, 2001, updated 2023.

[17] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC", NIST SP 800-38D, NIST, 2007.

[18] NIST, "The Keyed-Hash Message Authentication Code (HMAC)", FIPS 198-1, 2008.

[19] E. Barker, "Recommendation for Key Management: Part 1 – General", NIST SP 800-57 Part 1 Rev. 5, NIST, 2020.

[20] Microsoft, "Auditing for Azure SQL Database and Azure Synapse Analytics", https://learn.microsoft.com/en-us/azure/azure-sql/database/auditing-overview, accessed 7 November 2026.

[21] Amazon Web Services, "Monitoring Amazon RDS with Database Activity Streams", https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/DBActivityStreams.html, accessed 7 November 2026.

[22] pgAudit project, "PostgreSQL Audit Extension", https://www.pgaudit.org/, accessed 7 November 2026.

[23] Supabase, "PGAudit: Postgres Auditing", https://supabase.com/docs/guides/database/extensions/pgaudit, accessed 7 November 2026.

[24] Supabase, "Security at Supabase", https://supabase.com/security, accessed 7 November 2026.

[25] Amazon Web Services, "Securing connections to RDS for PostgreSQL with SSL/TLS", https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/PostgreSQL.Concepts.General.SSL.html, accessed 7 November 2026.

[26] Supabase, "Pricing", https://supabase.com/pricing, accessed 7 November 2026.

[27] Supabase, "Connect to your database", https://supabase.com/docs/guides/database/connecting-to-postgres, accessed 7 November 2026.

[28] Supabase, "Securing your API", https://supabase.com/docs/guides/api/securing-your-api, accessed 7 November 2026.